

Homework 1 - CSCI633

Evan Lin

02/17/2026

Question 1.

1) Give two examples of a combinatorial problem and describe them.

The Knapsack Problem: You have a backpack with a limited storage capacity. You're presented with several items, each with two properties: a weight and a value. The goal is to select the optimal combination of items to pack that maximizes the total value within the limit.

The Traveling Salesman Problem: A salesperson needs to visit a set of cities exactly once and return to the starting city. Given the distances between each pair of cities, the goal is to find the shortest possible route that visits all cities and returns to the origin.

2) What is most difficult when attempting to solve these problems?

The difficulty of combinatorial problem is that the number of possible solutions grows exponentially as the problem gets bigger. This phenomenon is also called combinatorial explosion.

For the **Knapsack Problem** with n items, there are 2^n possible combinations. With just 30 items, that's over 1 billion combinations to check.

For the **Traveling Salesman Problem** with n cities, there are $(n-1)!/2$ possible routes. So with just 20 cities, that's about 60 quadrillion possibilities.

This difficulty means that for realistically sized problems, we cannot guarantee finding the absolute optimal solution. Instead, we use metaheuristic algorithms that find good enough solutions in reasonable time without checking every possibility.

Question 2.

1) What is a metaheuristic? Answer this in your own words and in a meaningful/useful manner.

A metaheuristic is a algorithmic framework that provides strategies for solving complex optimization problems. The two major components are:

- **Intensification (exploitation):** searching in a local region to find a local optimum by focusing on promising areas where good solutions have already been found
- **Diversification (exploration):** generating diverse solutions to explore the search space on a global scale, ensuring that new regions are investigated

The combination of these two helps ensure that global optimality is achievable. Metaheuristics are typically stochastic and are designed to find good enough solutions to difficult problems without guaranteeing the best solution, but will be within a reasonable time.

2) What is the basic optimization problem setup (describe this formally as well as in your own words, defining any notation you introduce clearly).

The basic optimization problem can be setup as:

- **Minimize (or maximize):** $f(\mathbf{x})$
- **Subject to:** $\mathbf{x} \in \mathcal{X}$

where:

- $\mathbf{x} = (x_1, x_2, \dots, x_d)^T$ is a vector of decision variables
- $f(\mathbf{x})$ is the objective function (what we want to optimize)
- $\mathcal{X}$ is the feasible search space (all allowed solutions, often defined by constraints)

In simpler terms, we are trying to find the best possible solution from all allowed candidates, where best means either the smallest value or largest value of the objective function.

3) Furthermore, carefully explain the No Free Lunch (NFL) Theorem and how this applies to metaheuristic optimization (do not forget to appeal to and explain relevant pieces of its formal definition as it appears in the textbook and as we dissected in class).

The No Free Lunch (NFL) Theorem states that if any algorithm A outperforms another algorithm B in the search for a specific goal, then algorithm B will outperform A over other objective functions.

Formal Definition:

$$\sum_f P(s_m^y | f, m, A) = \sum_f P(s_m^y | f, m, B)$$

where $P(s_m^y | f, m, A)$ denotes the performance of algorithm A after m iterations on objective function f . This means that when averaged over all possible objective functions, the performance of any two algorithms is exactly the same. In other words, no algorithm is better than any other when considering every problem.

Applications to Metaheuristic Optimization: The NFL theorem tells us there is no single best metaheuristic that works for every problem. A metaheuristic that performs well on one type of problem may perform poorly on another. In practice, this means we should focus on finding algorithms that work well for the specific problems, rather than searching for a universal optimizer.

4) What are some incorrect conclusions that you should NOT draw from the NFL Theorem?.

One incorrect conclusion is saying all algorithms perform equally well on specific problems. The NFL theorem only applies when averaging over all possible problems. So for any specific problem, some algorithms will clearly outperform others.

Another wrong conclusion is that there is no point in developing new algorithms. We care about solving real-world problems, not all theoretical problems. New algorithms can be much better for the specific problem types that we actually see.

Question 3.

1) What is (deterministic) hill-climbing?

Hill-climbing is a search algorithm that tries to find the local maximum by moving into the highest value possible. It is a greedy algorithm because at each step, it tries to get as close to the peak as possible. Starting from an initial guess, the algorithm evaluates neighbors and moves to the highest value one.

This process is repeated until no neighbors has a higher value, which means it has reached a local peak. This algorithm is deterministic since if it starts again in the same spot, the result will not change.

2) Why do we need anything more complicated than hill-climbing? What are its deficiencies?

The main problem with hill-climbing is that it gets stuck in local peaks, because the algorithm is greedy and only accepts moves that improves the current situation.

3) How might we adapt hill-climbing to work better with respect to the exploration-exploitation trade-off?

We want a more sophisticated algorithms that balance exploitation with exploration. A common strategy is hill-climbing with random restart. By running the algorithm multiple times starting from different random initial points, it ensures that more peaks are reachable.

Question 4.

# Restarts	$f(x)_{\text{max Found}} (\mu \pm \sigma)$	# Iterations (mean)
N = 1	-0.08648953 $\pm$ 0.06631417	1000.00
N = 2	-0.05365193 $\pm$ 0.05963499	1000.00
N = 3	-0.03833419 $\pm$ 0.04220142	1000.00
N = 5	-0.01724928 $\pm$ 0.02713818	1000.00
N = 10	-0.00509919 $\pm$ 0.00488083	1000.00
N = 30	-0.00167490 $\pm$ 0.00120158	1000.00
N = 50	-0.00141682 $\pm$ 0.00120732	1000.00
N = 100	-0.00063830 $\pm$ 0.00040377	1000.00
N = 150	-0.00052621 $\pm$ 0.00032046	1000.00
N = 200	-0.00040562 $\pm$ 0.00021074	1000.00

Table 1: Performance of stochastic hill-climbing with random restarts on the negative Alpine function

Discussion

The results show that increasing restarts improves both solution quality and reliability. With a few restart (N=2), the algorithm often gets stuck in local max (mean = -0.05365, σ = 0.05963). As restarts increase, fitness approaches the global maximum of 0 and standard deviation decreases. At N=200, the algorithm finds solutions very close to optimal (mean = -0.0004056, σ = 0.00021). Iterations remain constant at 1000 since each restart runs for the maximum

allowed steps. This demonstrates the exploration-exploitation trade-off: more restarts (exploration) leads to better final solutions (exploitation).